

Laboratório 3 Constantes, Saltos, Subrotinas e Pilha

Atenção porque esse lab *é meio porrada*.

Chamadas de Sistema

O assembly RISC-V possui incorporada a instrução `ecall`, projetada para acessar as funções básicas do sistema (no PC, estaríamos falando das funções da UEFI/BIOS), tais como acesso a tela, teclado e arquivos. O RARS simula várias funções úteis com `syscalls`. Por exemplo, o trecho a seguir lê um inteiro do teclado e o coloca em `t0`. O usuário vai digitar dentro da aba “Run I/O” do painel inferior do RARS.

```
li      a7,5      # a7 é carregado para executar a chamada 5
ecall                   # faz a chamada (lê um inteiro da interface)
mv      t0,a0     # o inteiro foi colocado em a0; copiamos em t0
```

O que determina a função da `ecall` é o valor do registrador `a7`. Se `a7` vale 5, o sistema vai fazer a leitura de um número via teclado e vai escrever o valor lido no registrador `a0`. Veja a lista de chamadas de sistema na ajuda do RARS, incluindo um exemplo mais complexo ao final.



Faça um programa que lê um número inteiro pelo teclado e imprime o triplo dele no painel inferior de execução no RARS. Use a `syscall` de impressão de inteiros. Se quiser ser chique, imprima um espaço ou `return` antes do resultado final, usando a `syscall` de impressão de caracteres ASCII.

Constantes Negativas e Extensão de Sinal

Podemos assumir, por praticidade, que todos os números que lidamos no RISC-V são sinalizados, o que significa dizer que seus valores são representados em complemento de 2.

Obviamente, é preciso fazer *extensão de sinal*: `addi s0, zero, -1000` deve carregar `0xFFFFFC18` em `s0` (isso é -1000 em complemento de 2 com 32 bits), mas ao observar o opcode, vemos que a constante de 12 bits na instrução formato I (b31:b20) é `0xC18 = 1010 0001 10002`.



Fazer a “extensão de sinal” é apenas repetir o bit MSB para todos os bits à esquerda. Teste no RARS: carregue `s0` com -1000, carregue `s1` com 310 e some os dois, observando e explicando os valores hexadecimais envolvidos. Repita para números pequenos, como 5 e -2.

Contador de Programa e Saltos

O endereço da *próxima* instrução a ser executada por um processador é sempre guardado em um registrador chamado PC – *Program Counter*. Durante a execução de uma instrução comum, o PC é somado com 4 para apontar para a instrução seguinte (lembre: cada instrução de 32 bits ocupa 4 posições de memória no RISC-V RV32I).

Temos três tipos de codificação de saltos no RISC-V, dos quais veremos apenas o primeiro:

- *branches* condicionais `beq`, `bne`, `bge`, `blt` que usam o formato B;
- chamada de subrotina/função com `jal` que usa o formato J;
- retorno de subrotina/função com `jalr` que usa o formato I.

Note que o RISC-V *não possui salto com endereço absoluto*, muito comum em processadores. Neste tipo de salto, o endereço destino absoluto da memória está codificado como uma constante na instrução, que irá sobrescrever o PC.

Cálculo do “Delta” em *branches*

Os *branches* são codificados como instruções do tipo B, formato visto na seção seguinte. Eles são saltos relativos, ou seja, codificam um “delta” como constante da instrução, que será somado ao PC dizendo quantos endereços de memória devem ser saltados a partir da instrução atual.

O formato B reserva 12 bits para usar na constante deste “delta,” e esta constante é tratada como sinalizada (*signed*). É importante notar que o bit LSB de um endereço de instrução é *sempre* zero. Isto ocorre porque *todas as instruções no RISC-V começam obrigatoriamente em endereços pares* (estão alinhadas em endereços múltiplos de 2) para facilitar a decodificação, e $b_0 = 0$ para um número par.¹

Desta forma, temos 13 bits sinalizados (os 12 da constante mais o zero no LSB), o que nos dá limites entre $-2^{12} \leq \text{delta} < 2^{12} - 2$, ou seja, aproximadamente 4 kBytes de distância da instrução atual, para frente ou para trás na memória (ou +/-1024 instruções).²

O delta de 13 bits precisa ser convertido pelo circuito para 32 bits para ser adicionado ao PC de 32 bits (são condutores de um barramento e portas lógicas, não dá pra somar 13 bits com 32...). Isso significa *estender o sinal* do complemento de 2. Na prática, basta preencher os 19 bits restantes com o bit do sinal -- isso conserva o valor sinalizado original.

Por exemplo: $-4_{10} = 111111111100_2$ (13 bits) $\Rightarrow$ **1111111111111111** **1**111111111100₂ (32 bits). Se for positivo, colocar zeros à esquerda é trivial.

Veja um exemplo bobo:

<u>endereço</u> (hexa)	<u>instrução</u>
00400000	addi s1,zero,33
00400004	bne s1,zero,pulei # <= PC = 0x00400004
00400008	nop
0040000C	nop
00400010	pulei:addi s1,s1,1 # <= PC+12

O label “pulei:” está 3 instruções à frente do PC, considerando a instrução bne já lida pelo processador; isto significa que devemos pular 12 endereços com o bne (cada instrução gasta 4 bytes). Podemos exprimir 12_{10} em 13 bits e cortar o LSB: $0\ 0000\ 0000\ 110\mathbf{0}_2 \Rightarrow 0000\ 0000\ 0110_2$, obtendo os 12 bits da constante. Na execução, o RISC-V vai recolocar este 0 LSB, estender o sinal e somar este número ao PC atual.

Codificação no Formato B

As instruções do tipo B, que codificam *branches*, possuem um esquema estranho:

	31	25	24	20	19	15	14	12	11	7	6	0
Formato B	imm1 (7)	rs2 (5)	rs1 (5)	funct3 (3)	imm2 (5)	opcode (7)						

A constante imediata *imm* (o delta de 13 bits calculado anteriormente) é codificada em pedaços, da seguinte forma:

1. $\text{imm1} = \text{imm}[12:10:5]$
2. $\text{imm2} = \text{imm}[4:1:11]$

sendo | a concatenação. Note que $\text{imm}[0] = 0$ sempre (*imm* é par) e que os bits $\text{imm}[31:13]$ são apenas a extensão de sinal da constante.

Ou seja, os bits estão embaralhados! ³

1 Poderiam ser múltiplos de 4 no RV32I, mas existe o subset de instruções compactadas de 16 bits etc, etc...

2 É $2^{12}-2$ ao invés de $2^{12}-1 = 4094$ porque o último bit LSB está sempre em zero ($0\ 1111\ 1111\ 1110_2 = 4094$).

3 A razão para isso é diminuir os multiplexadores na decodificação de instruções, acelerando o circuito. Isso ocorre porque os bits [10:5] e os bits [4:1] são os mesmos do formato S, e portanto o processador não precisa diferenciar estes dois formatos. O chuncho com a troca do b31 e b12 tem a ver com extensão de sinal e fan-out excessivo de portas lógicas, e também é para acelerar. E santo Stack Overflow nessas horas:

<https://stackoverflow.com/questions/39427092/risc-v-immediate-encoding-variants>

Exemplo: para codificar um branch no qual calculamos um $\text{delta imm} = 0x170C$ de 13 bits = **1**0**111**0000**1100**₂:

1. $\text{imm1} = \text{1111000}$;

2. $\text{imm2} = \text{01100}$,

com o último zero 0 da constante descartado. Os demais campos da instrução são triviais.

O circuito do microprocessador vai fazer o caminho inverso, reconstruindo a constante original a partir do código binário da instrução. Isso pode ser obtido facilmente com a equação $\text{imm} = \text{extensão do sinal} \mid \text{imm1}[6] \mid \text{imm2}[0] \mid \text{imm1}[5:0] \mid \text{imm2}[4:1] \mid 0$. Este valor imm será somado ao PC para executar o branch.

▶	Compile no RARS o programa bobo abaixo e explique <i>cada bit</i> da codificação binária resultante. Assuma que o endereço de <code>start</code> é <code>0x0040 0000</code>. <pre>start: beq zero, zero, final nop nop bge zero, zero, start final: bne s1, s2, final</pre>
---	--

Subrotinas

Funções em *assembly* são usualmente chamadas de subrotinas e possuem várias convenções para interoperabilidade, já que estamos mexendo diretamente com os detalhes do processador neste nível.

Quando uma função é encerrada, a execução deve voltar ao ponto em que ela foi chamada. Mas este ponto pode variar, já que a função pode ser chamada diversas vezes, de diversos lugares do programa. Como saber qual o endereço de retorno? A estratégia do RISC-V é armazenar este endereço no registrador `ra` (ou `x1`, é o *return address*). Veja:

```
main:    jal    nada # chama a subrotina; "volta" é guardado em ra
volta:   li     s3, 321
          jal    nada # chama "nada"; vai armazenar "fim" em ra
fim:     li     a7, 10 # finalização elegante; syscall 10
          ecall    # retorna para o sistema operacional

# --- Subrotina "nada" ---
nada:    li     t0, 777
          jr     ra # pula para o endereço dado por ra (volta ou fim)
```

A instrução `jal` (*jump-and-link*) é um salto modificado que pula para o destino e guarda automaticamente o valor do endereço da instrução seguinte no registrador `ra` ($\text{PC}+4$, é o ponto de retorno da chamada)⁴. As subrotinas devem portanto terminar com `jr ra` (*jump register*, salta para o endereço indicado pelo registrador `ra`) para saltar *explicitamente* para o ponto de retorno da chamada⁵.

Parâmetros de Subrotinas

Para usarmos parâmetros e retornarmos resultados das funções, usamos os registradores padrão `a0~a7` para os argumentos, nesta ordem, e novamente `a0` para o resultado (com `a1` se o resultado

4 Tecnicamente `jal destino` é uma pseudo-instrução, baseada na instrução nativa `jal reg, destino`, que grava o endereço de retorno no registrador `reg` especificado. Apenas ocorre que, se omitido o registrador, assume-se que é `ra`. Mas isso é detalhe demais para nossa disciplina.

5 Novamente uma pseudo-instrução: a instrução nativa é `jalr reg1, reg2, delta`, que guarda o endereço da instrução seguinte em `reg1` e salta para o endereço dado por `reg2+delta`. A tal instrução *jump register* `jr reg` é codificada como `jalr zero, reg, 0x0`, ou seja, grava o endereço de retorno no registrador constante zero (equivale a descartar essa informação) e salta para o endereço dado por `reg+0x0`, que é o retorno da chamada.

for de 64 bits). Veja a subrotina “prod”, que tem como resultado o produto de seus dois parâmetros:

```
main: li    a0,3
      li    a1,2      # carrega parâmetros (3,2) e chama função
      jal   prod
      mv    s0,a0      # copia o resultado de a0 para s0
      li    a7,10
      ecall          # retorna para o sistema operacional

prod: mul   a0,a0,a1   # apenas multiplica os parâmetros
      jr    ra         # retorna; resultado (6) em a0
```

Note que isso é apenas uma convenção⁶, e apesar de ser uma péssima ideia desobedecê-la, isso é possível.



Construa um programa que obtém um valor *válido* de temperatura pelo teclado, o converte de Celsius para Fahrenheit e o imprime, usando no mínimo duas funções auxiliares. *Utilize obrigatoriamente as convenções descritas anteriormente.* Use números inteiros.

Pilha

A pilha é uma área de memória destinada a guardar dados temporários. É uma estrutura LIFO (*Last In First Out*) que costuma ser usada por processadores para armazenar variáveis locais⁷ e endereços de retorno de funções. O endereço do topo da pilha na memória é dado pelo registrador `sp` (*Stack Pointer*). As operações são chamadas *push* (guardar dado) e *pop* (recuperar dado), e podem ser vistas na próxima seção.

As convenções historicamente mais populares assumem que a pilha aumenta em direção ao endereço `0x0000 0000` (ou seja, `sp` vai diminuindo, *descending*) e `sp` aponta para o último elemento que foi empilhado (*full stack*). Em outras convenções, a pilha pode crescer em direção ao fim da memória e `sp` pode apontar para o próximo espaço a ser utilizado, por exemplo.

Funções que Chamam Outras Funções

Se uma função `func1` chama outra com nome `func2`, teremos dois endereços de retorno a armazenar, um para cada chamada, mas apenas um `ra`. A saída é preservar temporariamente o primeiro endereço de retorno em algum lugar, convencionalmente na pilha. Veja uma função `func1` que apenas chama corretamente outra função `func2`:

```
func1: addi sp,sp,-4 # reserva espaço na pilha para um dado
      sw    ra,0(sp) # guarda o ra original no topo da pilha

      jal   func2     # isso vai destruir o valor original de ra!

      lw    ra,0(sp)  # recupera valor original guardado na pilha
      addi sp,sp,4    # libera o espaço reservado para o ra
      jr    ra        # retorna para o endereço correto

func2: jr ra
```



Simule o programa anterior e observe os valores de `ra`, `pc` e `sp`.

6 Estas convenções fazem parte de uma ABI – *Application Binary Interface*. Uma ABI define várias coisas, entre elas o comportamento padrão de funções, para que os *linkers* (linkeditores) consigam ligar módulos de diferentes linguagens num mesmo programa-alvo. Poderíamos, por exemplo, usar um módulo em C, outro em *assembly* e um terceiro em Java, compilando tudo para um único executável.

7 Variáveis com *storage class* automática, isso é toda uma história na disciplina de compiladores.

Isso resolve um problema; mas e os outros registradores? Suponha agora que `func1` vá precisar de 3 registradores e `func2` de outros 4. Como garantir que `func2` não vai destruir nada importante? A solução é uma convenção simples: *uma função pode destruir os valores de `t0` a `t6` (temporários) e `a0` a `a7` (argumentos), mas não pode destruir nenhum outro, em especial `s0` a `s11` (salvos).*

A regra é: a função chamadora empilha/recupera quaisquer registradores `t`'s ou `a`'s que precisem ser mantidos após a chamada; a função chamada empilha/recupera os registradores `ra` e `s`'s que for destruir.⁸

►	Construa um programa que calcula $y=3x^5+2x^3-6x$ para um valor de <code>x</code> entrado pelo teclado. A <code>main</code> deverá pegar o valor, chamar uma função que calcula <code>y</code> e imprimir o resultado. A função que calcula <code>y</code> deverá obrigatoriamente chamar uma função <code>pow</code> , que eleva um número <code>a0</code> a outro <code>a1</code> . É obrigatório utilizar as convenções descritas anteriormente.
---	--

►	Utilizando uma função hipotética já pronta chamada <code>cemseno</code> que calcula o valor de $100*\text{seno}(a0)$, que infelizmente destrói todos os <code>t</code> 's e <code>a</code> 's no processo, faça uma subrotina que devolve como resultado $a0+\text{cemseno}(3*a1)-\text{cemseno}(3*a0)+a2*a1$. É obrigatório utilizar as convenções descritas anteriormente.
---	---

[Opcional] Diretivas e Constantes em Memória

Para inserirmos constantes no nosso programa de forma elegante, utilizamos *diretivas assembly*, que são linhas de código que não resultam em instruções RISC-V quando compiladas. Começam com ponto decimal, como abaixo.

- `.asciz` (com apenas um `i`) serve para especificar uma string terminada em zero (caracteres de *um byte cada*);
- `.word` inclui uma lista de valores de 32 bits, a partir do próximo endereço múltiplo de 4.

Note que precisamos definir seções no *assembly* para o montador se achar, usando:

- `.data` inicia uma seção de dados, ou seja, indica que as linhas a seguir são apenas dados (constantes ou espaço reservado para elas);
- `.text` inicia uma seção de programa, ou seja, indica que as linhas a seguir são instruções RISC-V. É o *default*.

Por exemplo, abaixo temos em amarelo constantes a serem usadas posteriormente e, em verde, instruções que serão executadas.

```

.data                                     # início do segmento (trecho) dados
frase: .asciz "Frase a imprimir"         # \0 inserido automaticamente
dados: .word 34,-12,0,0x5FFE0D23         # 4 words de 32 bits

.text                                    # início de segmento de programa
li    t1,0x12345678
add   s1,s2,s3

```

Vamos compilar essas linhas e observar a memória durante a execução deste programa (note que ainda não usamos os dados pra nada). Os números 32, 12, 0 e 0x5FFE0D23 eu inventei na hora, só pra ocupar espaço. Vou usar um `print` do MARS MIPS ao invés do `RARS RISC-V`, mas é igual.

Lembre-se que cada endereço da RAM ocupa um byte mas *cada dado possui 32 bits* e, portanto, ocupa 4 endereços.

⁸ Essas operações acabam sendo simplificadas pelo uso do modo indexado de endereçamento (a constante imediata que representa um delta nos *loads* e *stores*).


```
# Ex: como ler a RAM no endereço de "dados"?
la s1,dados      # carrega o endereço dos dados em $s1
lw s2,0(s1)      # carrega da memória o valor 34 em $s2
# Como ler de "dados:"+12, ou seja, o quarto número?
lw s3,12(s1)     # lê 0x5FFE0D23 do endereço 1210+0x10010014
```

A pseudoinstrução `la` (*load address*) carrega o endereço de um *label* no registrador indicado. Como vimos na captura de tela anterior, `dados:` foi colocado pelo RARS no endereço `0x1001 0014`. Isso significa que:

```
la a0,frase      aqui equivale a      li a0,0x10010000
la s1,dados      aqui equivale a      li s1,0x10010014
```

A chamada do `ecall` com `a7` valendo 4 irá imprimir uma string na tela⁹. Para funcionar, o ponteiro para a string deve estar em `a0`, ou seja, `a0` indica onde começa a string.

► Faça um programa que obtém vinte números do teclado e os imprime na tela na ordem inversa à de entrada. Pode usar a diretiva `.space` se quiser, mas `.word` serve também.

Dicas: pesquise as chamadas na lista de `syscalls` para saber como ler e imprimir um número. Se quiser, use um `.word` com 20 zeros como se fosse um vetor inicializado com zero.

Exercícios (entregar três deles no Moodle, um deles de funções)

1. Faça um programa que carrega as seguintes constantes nos registradores `s0~s5`: -7,15,-123,+123,-500,-23. O programa deve somar todos estes valores e guardar o resultado final no registrador `a0`. *Determine no papel* o código de máquina para todas as instruções nativas envolvidas. (Estou supondo `addi`). Determine o resultado final das somas em binário e hexadecimal.
2. Para *cada um* dos branches no programa abaixo, *determine manualmente*: a) a constante “delta” que indica quantos endereços serão saltados, expressa em decimal sinalizado; b) os campos `imm1` e `imm2` usados para expressar esta constante no formato B do assembly; e c) os códigos de máquinas hexadecimais finais que exprimem estas constantes. Pode ignorar as demais instruções. (Temos, portanto, 15 itens neste exercício!).

```
                beq s1,s2,label5
                bne t0,t5,label4
label1:         nop
label2:         nop
                nop
                nop
label3:         nop
                bge s4,t3,label2
label4:         blt t1,s1,label3
                nop
label5:         bne s2,s2,label1
```

3. Faça um programa que lê 10 números inteiros do teclado e imprime na tela a soma de todos eles. Pesquise as chamadas na lista de `syscalls` para saber como ler e imprimir um número.
4. Faça e teste uma função *recursiva* que calcula o fatorial de um número, *obedecendo às convenções descritas*. Sugiro me perguntar se está correta.
5. Construa uma função chamada *shoobedoo* que calcula e devolve como resultado o valor de `a0+crunch(a1*a2)+3*a3-crunch(a4)`. Assuma que *crunch* é uma função que chama uma

⁹ A saída de dados na tela também se dá na aba “Run I/O” do painel inferior do RARS (abaixo da janela de conteúdo de memória, quando estivermos simulando um programa).

função chamada *clorofila* que recebe dois parâmetros. Mostre o código da função *shoobeedoo* e calcule quantos bytes da pilha serão utilizados numa chamada a ela.

6. Construa e teste uma função que retorna como resultado a soma de oito valores passados a ela como parâmetros. Utilize a convenção do C. É obrigatório utilizar as convenções descritas anteriormente.

Discussão (apenas para refletir)

1. Há um comando chamado *nop* (*no operation*) no *assembly* RISC-V, que não faz nada. Ele está presente na maioria dos microprocessadores. Para que ele serve?
2. Note que alguns processadores sempre salvam o endereço de retorno na pilha, mesmo que a subrotina não chame nenhuma outra (seja uma “folha”). Tente argumentar a favor disso.
3. O x86 da Intel/AMD sempre teve poucos registradores de uso geral. Em que circunstâncias isso é desejável, ao invés de termos muitos deles? Por que não podemos ter centenas deles?
4. Algumas famílias de microcontroladores (ISA's) como do 8051 ou PIC *não possuem* registradores de uso geral explícitos, além do acumulador, que é especial, usando ao invés deles referências à memória RAM interna do processador (de 128 a 512 bytes). Isso é obviamente razoável (já que ainda se vendem 8051's). Por quê?